

Chapter 9. Delegates, Lambdas, and Events

The most common way to use an API is to invoke the methods and properties its classes provide, but sometimes things need to work in reverse—the API may need to call your code, an operation often described as a *callback*. In [Chapter 5](#), I showed the search features offered by arrays and lists. To use these, I wrote a method that returned `true` when its argument met my criteria, and the relevant APIs called my method for each item they inspected. Not all callbacks are this immediate. Asynchronous APIs can call a method in our code when long-running work completes. In a client-side application, I want my code to run when the user interacts with certain visual elements in particular ways, such as clicking a button.

Interfaces and virtual methods can enable callbacks. In [Chapter 4](#), I showed the `IComparer<T>` interface, which defines a single `CompareTo` method. This is called by methods like `Array.Sort` when we want a customized sort ordering. You could imagine a UI framework that defined an `IClickHandler` interface with a `Click` method, and perhaps also `DoubleClick`. The framework could require us to implement this interface if we want to be notified of button clicks.

In fact, none of .NET's UI frameworks use the interface-based approach, because it gets cumbersome when you need multiple kinds of callback. Single- and double-clicks are the tip of the iceberg for user interactions—in WPF applications, each UI element can provide over 100 kinds of notifications. Most of the time, you need to handle only one or two events from any particular element, so an interface with 100 methods to implement would be annoying.

Splitting notifications across multiple interfaces could mitigate this inconvenience. Default interface implementations could help, because it would make it possible to provide default, empty implementations for all call-

backs, meaning we'd need to override only the ones we were interested in. (Neither .NET Standard 2.0 nor .NET Framework support this language feature, but a library targeting those could supply a base class with virtual methods instead.) But even with these refinements, there's a serious drawback with this object-oriented approach. Imagine a UI with four buttons. In a hypothetical UI framework that used the approach I've just described, if you wanted each button to have its own click handler, you'd need four distinct implementations of the `IClickHandler` interface. A single class can implement any particular interface only once, so you'd need to write four classes. That seems very cumbersome when all we really want to do is tell a button to call a particular method when clicked.

C# provides a much simpler solution in the form of a *delegate*, which is a reference to a method. If you want a library to call your code back for any reason, you will normally just pass a delegate referring to the method you'd like it to call. I showed an example of that in [Chapter 5](#), which I've reproduced in [Example 9-1](#). This finds the index of the first element in an `int[]` array with a nonzero value.

Example 9-1. Searching an array using a delegate

```
public static int GetIndexOffirstNonEmptyBin(int[] bins)
    => Array.FindIndex(bins, IsNonZero);

private static bool IsNonZero(int value) => value != 0;
```

At first glance, this seems straightforward: the second parameter to `Array.FindIndex` requires a method that it can call to ask whether a particular element is a match, so I passed my `IsNonZero` method as an argument. But what does it really mean to pass a method, and how does this fit in with .NET's type system, the CTS?

Delegate Types

[Example 9-2](#) shows the declaration of the `FindIndex` method used in [Example 9-1](#). The first parameter is the array to be searched, but it's the second one we're interested in—that's where I passed a method.

Example 9-2. Method with a delegate parameter

```
public static int FindIndex<T>(
    T[] array,
    Predicate<T> match)
```

The method's second parameter's type is `Predicate<T>`, where `T` is the array element type, and since [Example 9-1](#) uses an `int[]`, that will be a `Predicate<int>`. (In case you don't have a background in either formal logic or computer science, this type uses the word *predicate* in the sense of a function that determines whether something is true or false. For example, you could have a predicate that tells you whether a number is even. Predicates are often used in this kind of filtering operation.) [Example 9-3](#) shows how this type is defined. This is the whole of the definition, not an excerpt; if you wanted to write a type that was equivalent to `Predicate<T>`, that's all you'd need to write.

Example 9-3. The `Predicate<T>` delegate type

```
public delegate bool Predicate<in T>(T obj);
```

Breaking [Example 9-3](#) down, we begin as usual with the accessibility, and we can use all the same keywords we could for other types, such as `public` or `internal`. (Like any type, delegate types can optionally be nested inside some other type, in which case you can also use `private` or `protected`.) Next is the `delegate` keyword, which tells the C# compiler that we're defining a delegate type. The rest of the definition looks, not coincidentally, just like a method declaration. We have a return type of `bool`. You put the delegate type name where you'd normally see the method name. The angle brackets indicate that this is a generic type with a single type parameter `T`, and the `in` keyword indicates that `T` is contravariant. Finally, the method signature has a single parameter of that type.

TIP

The use of contravariance here lets you use a predicate that is more general than would otherwise be required. For example, because all values of type `string` are compatible with the type `object`, all values of `Predicate<object>` are compatible with the type `Predicate<string>`. Or to put that informally, if an API needs a method that inspects a `string`, it will work perfectly well if you pass it a method that is able to inspect any `object`. [Chapter 6](#) described contravariance in detail.

Delegate types are special in .NET, and they work quite differently than classes or structs. The compiler generates a superficially normal-looking type definition with various members that we'll look at in more detail later, but the members are all empty—C# produces no IL for any of them. The CLR provides the implementation at runtime.

A delegate type's parameter list can make arguments optional by defining default values, just like with a normal method declaration, as [Example 9-4](#) shows. I'll show how to invoke a delegate shortly, but optional arguments are handled in exactly the same way as when invoking methods.

Similarly, you can define a delegate type that has the `params` keyword on the final parameter (which must then be an array type) to enable variable numbers of arguments.

Example 9-4. A delegate type with one optional argument

```
public delegate void Log(string message, string? source = "");
```

Instances of delegate types are usually just called delegates, and they refer to methods. A method is compatible with (i.e., can be referred to by an instance of) a particular delegate type if its signature matches. The `IsNonZero` method in [Example 9-1](#) takes an `int` and returns a `bool`, so it is compatible with `Predicate<int>`. The match does not have to be precise. If implicit reference conversions are available for parameter types, you can use a more general method. (Although this may sound very similar to the upshot of `T` being contravariant, this is a subtly different issue. `T` being contravariant in `Predicate<T>` determines what types an existing instance of `Predicate<T>` can be converted to. This is separate from the rules around whether you can construct a new delegate of some specific type from a particular method: the signature matching rules I'm

now describing apply even for nongeneric delegates, and for generic delegates with invariant type parameters.) For example, a method with a return type of `bool`, and a single parameter of type `object`, would be compatible with `Predicate<object>`, but because such a method can accept `string` arguments, it would also be compatible with `Predicate<string>`. (It would not be compatible with `Predicate<int>`, because there's no implicit reference conversion from `int` to `object`. There's an implicit conversion, but it's a boxing conversion, not a reference conversion.)

Creating a Delegate

The simplest way to create a delegate is to write just the method name.

[Example 9-5](#) declares a variable, `p`, and initializes it with the `IsNonZero` method from [Example 9-1](#). (This code requires `IsNonZero` to be in scope, so we could only write this inside the same class.)

Example 9-5. Creating a delegate

```
var p = IsNonZero;
```

This example says nothing about the particular delegate type required, which causes the compiler to pick from one of a couple of families of generic types that I'll be describing later in this chapter. In the unusual cases where you can't use those, it will define a type for you. In this case, it will use `Func<int, bool>`, reflecting the fact that `IsNonZero` is a method that takes an `int` and returns a `bool`. This is a reasonable choice, but what if I wanted to use the `Predicate<int>` type because I'm planning to pass it to `Array.FindIndex`, as in [Example 9-1](#)? If you don't want the compiler's default choice, you can use the `new` keyword, as [Example 9-6](#) shows. This lets you state the type, and where you'd normally pass constructor arguments, you can supply the name of a compatible method.

Example 9-6. Constructing a delegate

```
var p = new Predicate<int>(IsNonZero);
```

In practice, we rarely use `new` for delegates. It's necessary only in cases where the compiler will not infer the right delegate type. Typically, the compiler can work it out from context. [Example 9-7](#) declares a variable with an explicit type, so the compiler knows a `Predicate<int>` is required—we don't need to use `new` here. This has the same effect as [Example 9-6](#), although thanks to a feature added in C# 11.0 it will be slightly more efficient in cases where the code runs many times. The compiler generates a hidden static field to hold the delegate, meaning construction only has to happen the first time this line of code runs; it will reuse the same delegate on all subsequent occasions.

Example 9-7. Implicit delegate construction

```
Predicate<int> p = IsNonZero;
```

That still mentions the delegate type name explicitly, but often we don't even need to do that. [Example 9-1](#) correctly determined that `IsNonZero` needed to be turned into a `Predicate<int>` without us needing to say so. The compiler knows that the second argument to `FindIndex` is `Predicate<T>`, and because we supplied a first argument of type `int[]`, it deduced that `T` is `int`, so it knows the second argument's full type is `Predicate<int>`. Having worked that out, it uses the same built-in implicit conversion rules to construct the delegate as [Example 9-7](#). So when you pass a delegate to a method, the compiler can normally work out the right type by itself.

When code refers to a method by name like this, the name is technically called a *method group*, because multiple overloads may exist for a single name. The compiler narrows this down by looking for the best possible match in a similar way to how it chooses an overload when you invoke a method. As with method invocation, it is possible that there will be either no matches or multiple equally good matches, and in those cases the compiler will produce an error.

Method groups can take several forms. In the examples shown so far, I have used an unqualified method name, which works only when the method in question is in scope. If you want to refer to a static method defined in some other class, you would need to qualify it with the class name, as [Example 9-8](#) shows.

Example 9-8. Delegates referring to methods in another class

```
internal static class Program
{
    static void Main()
    {
        Predicate<int> p1 = Tests.IsGreaterThanZero;
        Predicate<int> p2 = Tests.IsLessThanZero;
    }
}

internal class Tests
{
    public static bool IsGreaterThanZero(int value) => value > 0;

    public static bool IsLessThanZero(int value) => value < 0;
}
```

Delegates don't have to refer to static methods. They can refer to an instance method. There are a couple of ways you can make that happen. One is simply to refer to an instance method by name from a context in which that method is in scope. The `GetIsGreaterThanPredicate` method in [Example 9-9](#) returns a delegate that refers to `IsGreaterThan`. Both are instance methods, so they can be used only with an object reference, but `GetIsGreaterThanPredicate` has an implicit `this` reference, and the compiler automatically provides that to the delegate that it implicitly creates. (This prevents the compiler from using the optimization introduced in C# 11.0—in these cases it can't create a hidden static field to cache the delegate because each delegate will refer to a specific instance of `ThresholdComparer`. In principle it could create a hidden nonstatic field instead, but it does not. Adding an extra field to every instance of a type has a higher cost than a single static field, so there's a significant risk that this would have a negative effect.)

Example 9-9. Implicit instance delegate

```
public class ThresholdComparer
{
    public required int Threshold { get; init; }
```

```

    public bool IsGreaterThan(int value) => value > Threshold;

    public Predicate<int> GetIsGreaterThanPredicate() => IsGreaterThan;
}

```

Alternatively, you can be explicit about which instance you want.

[Example 9-10](#) creates three instances of the `ThresholdComparer` class from [Example 9-9](#), and then creates three delegates referring to the `IsGreaterThan` method, one for each instance.

Example 9-10. Explicit instance delegate

```

var zeroThreshold = new ThresholdComparer { Threshold = 0 };
var tenThreshold = new ThresholdComparer { Threshold = 10 };
var hundredThreshold = new ThresholdComparer { Threshold = 100 };

Predicate<int> greaterThanZero = zeroThreshold.IsGreaterThan;
Predicate<int> greaterThanTen = tenThreshold.IsGreaterThan;
Predicate<int> greaterThanOneHundred = hundredThreshold.IsGreaterThan;

```

You don't have to limit yourself to simple expressions of the form `variableName.MethodName`. You can take any expression that evaluates to an object reference, and then just append `.MethodName`; if the object has one or more methods called `MethodName`, that will be a valid method group.

NOTE

You can define delegate types with any number of parameters. For example, the runtime libraries define `Comparison<T>`, which compares two items, and therefore takes two arguments (both of type `T`).

C# will not let you create a delegate that refers to an instance method without specifying either implicitly or explicitly which instance you mean, and it will always initialize the delegate with that instance.

NOTE

When you pass a delegate to some other code, that code does not need to know whether the delegate's target is a static or an instance method. And for instance methods, the code that uses the delegate does not supply the instance. Delegates that refer to instance methods always know which instance they refer to, as well as which method.

There's another way to create a delegate that can be useful if you do not necessarily know which method or object you will use until runtime: you can use the reflection API (which I will explain in detail in [Chapter 13](#)). First, you obtain a `MethodInfo`, an object representing a particular method. Then you call its `CreateDelegate<TDelegate>` method, specifying the delegate type as a type argument and, where required, passing the target object. (If you're creating a delegate referring to a static method, there is no target object, so there's an overload that takes only the delegate type.) This will create a delegate referring to whichever method the `MethodInfo` instance identifies. [Example 9-11](#) uses this technique. It obtains a `Type` object (also part of the reflection API; it's a way to refer to a particular type) representing the `ThresholdComparer` class. Next, it asks it for a `MethodInfo` representing the `IsGreaterThan` method. On this, it calls the overload of `CreateDelegate` that takes the delegate type and the target instance.

Example 9-11. `CreateDelegate`

```
MethodInfo m = typeof(ThresholdComparer).GetMethod("IsGreaterThan");  
var greaterThanZero = m.CreateDelegate<Predicate<int>>(zeroThreshold);
```

There is another way to perform the same job: the `Delegate` type has a static `CreateDelegate` method, which avoids the need to obtain the `MethodInfo`. You pass it two `Type` objects—the delegate type and the type defining the target method—and also the method name. If you already have a `MethodInfo` in hand, you may as well use that, but if all you have is the name, this alternative is more convenient.

WARNING

Selecting a delegate target by name at runtime is likely not to work if you build a trimmed self-contained executable (e.g., if you're using Native AOT) as described in [Chapter 12. Example 9-11](#) specifies the target type and method names directly, so the build tools can deduce that they must not remove the `ThresholdComparer.IsGreaterThan` method in this case, but in general, code that uses these kinds of dynamic techniques to choose the target at runtime is typically incompatible with trimming.

To summarize what we've seen so far, a delegate identifies a specific function, and if that's an instance function, the delegate also contains an object reference. But some delegates do more.

Multicast Delegates

If you look at any delegate type with a reverse-engineering tool such as ILDASM,¹ you'll see that whether it's a type supplied by the runtime libraries or one you've defined yourself, it derives from a base type called `MulticastDelegate`. As the name suggests, this means delegates can refer to more than one method. This is mostly of interest in notification scenarios where you may need to invoke multiple methods when some event occurs. However, all delegates support this whether you need it or not.

Even delegates with non-`void` return types derive from `MulticastDelegate`. That doesn't usually make much sense. For example, code that requires a `Predicate<T>` will normally inspect the return value. `Array.FindIndex` uses it to find out whether an element matches our search criteria. If a single delegate refers to multiple methods, what's `FindIndex` supposed to do with multiple return values? It just accepts the default behavior, which is to execute all the methods but to ignore the return values of all except the final method that runs. (It's possible to write code to provide special handling for multicast delegates, but `FindIndex` does not.)

The multicast feature is available through the `Delegate` class's static `Combine` method. This takes any two delegates and returns a single delegate. When the resulting delegate is invoked, it is as though you invoked the two original delegates one after the other. This works even when the

delegates you pass to `Combine` already refer to multiple methods—you can chain together ever larger multicast delegates. If the same method is referred to in both arguments, the resulting combined delegate will invoke it twice.

NOTE

Delegate combination always produces a new delegate. And the `Combine` method doesn't modify either of the delegates you pass it.

In fact, we rarely call `Delegate.Combine` explicitly, because C# has built-in support for combining delegates. You can use the `+` or `+=` operators. [Example 9-12](#) shows both, combining the three delegates from [Example 9-10](#) into a single multicast delegate. The two resulting delegates are equivalent—this just shows two ways of writing the same thing. Both cases compile into a couple of calls to `Delegate.Combine`.

Example 9-12. Combining delegates

```
Predicate<int> megaPredicate1 =  
    greaterThanZero + greaterThanTen + greaterThanOneHundred;  
  
Predicate<int> megaPredicate2 = greaterThanZero;  
megaPredicate2 += greaterThanTen;  
megaPredicate2 += greaterThanOneHundred;
```

You can also use the `-` or `--` operators, which produce a new delegate that is a copy of the first operand but with its last reference to the method referred to by the second operand removed. As you might guess, this turns into a call to `Delegate.Remove`.

Invoking a Delegate

So far, I've shown how to create a delegate, but what if you're writing your own API that needs to call back into a method supplied by your caller? First, you would need to pick a delegate type. You could use one supplied by the runtime libraries, or, if necessary, you can define your own. Then, you can use this delegate type for a method parameter or a

property. [Example 9-13](#) shows what to do when you want to call the method (or methods) the delegate refers to.

Example 9-13. Invoking a delegate

```
public static void CallMeRightBack(Predicate<int> userCallback)
{
    bool result = userCallback(42);
    Console.WriteLine(result);
}
```

As this not terribly realistic example shows, you can use an argument of delegate type as though it were a function. This also works for local variables, fields, and properties. In fact, any expression that produces a delegate can be followed by an argument list in parentheses. The compiler will generate code that invokes the delegate. If the delegate has a non-void return type, the invocation expression's value will be whatever the underlying method returns (or, in the case of a delegate referring to multiple methods, whatever the final method returns).

Although delegates are special types with runtime-generated code, there is ultimately nothing magical about invoking them. Invoking a delegate with a single target method works as though your code had called the target method in the conventional way. Invoking a multicast delegate is just like calling each of its target methods in turn. In either case, calls happen on the same thread, and exceptions propagate out of methods that were invoked via a delegate in exactly the same way as they do when you invoke the method directly.

If you want to get all the return values from a multicast delegate, you can take control of the invocation process. Delegates offer a `GetInvocationList` method, which returns an array containing a single-method delegate for each of the methods to which the original multicast delegate refers. If you call this on a normal, nonmulticast delegate, this list will contain just that one delegate, but if the multicast feature is being exploited, you could then loop over the array, invoking each in turn.

There is one more way to invoke a delegate that is occasionally useful. The base `Delegate` class provides a `DynamicInvoke` method. You can call

this on a delegate of any type without needing to know at compile time exactly what arguments are required. It takes a `params` array of type `object[]`, so you can pass any number of arguments. It will verify the number and type of arguments at runtime. This can enable certain late-binding scenarios. The intrinsic language features enabled by the `dynamic` keyword (discussed in [Chapter 2](#)) are more comprehensive, but they are slightly more heavyweight due to the extra flexibility, so if `DynamicInvoke` does precisely what you need, it is the better choice. (As with the dynamic delegate creation mechanisms we saw earlier, this technique is not a good fit with trimming, so you should avoid this if you want to use Native AOT.)

Common Delegate Types

The runtime libraries provide several useful delegate types, and you will often be able to use these instead of needing to define your own. For example, there is a set of generic delegates named `Action` with varying numbers of type parameters. These all follow a common pattern: for each type parameter, there's a single method parameter of that type.

[Example 9-14](#) shows the first four, including the zero-argument form.

Example 9-14. The first few `Action` delegates

```
public delegate void Action();  
public delegate void Action<in T1>(T1 arg1);  
public delegate void Action<in T1, in T2 >(T1 arg1, T2 arg2);  
public delegate void Action<in T1, in T2, in T3>(T1 arg1, T2 arg2, T3 arg3);
```

Although this is clearly an open-ended concept—you could imagine delegates of this form with any number of parameters—the CTS does not provide a way to define this sort of type as a pattern, so the runtime libraries have to define each form as a separate type. Consequently, there is no 200-parameter form of `Action`. The largest has 16 parameters.

The obvious limitation with `Action` is that these types have a `void` return type, so they cannot refer to methods that return values. But there's a similar family of delegate types, `Func`, that allows any return type.

[Example 9-15](#) shows the first few delegates in this family, and as you can see, they're pretty similar to `Action`. They just get an additional final

type parameter, `TResult`, which specifies the return type. As with `Action<T>`, these go up to 16 parameters.

Example 9-15. The first few `Func` delegates

```
public delegate TResult Func<out TResult>();  
public delegate TResult Func<in T1, out TResult>(T1 arg1);  
public delegate TResult Func<in T1, in T2, out TResult>(T1 arg1, T2 arg2);  
public delegate TResult Func<in T1, in T2, in T3, out TResult>(T1 arg1, T2 arg2, T3 arg3);
```

These `Action` and `Func` types are the ones C# will use as the *natural* type of a delegate expression, when possible. You saw this earlier in [Example 9-5](#), when, in the absence of any other direction, the compiler picked `Func<int, bool>`. It will use the `Action` family for methods that have a `void` return type.

These two families of delegates would appear to have most requirements covered. Unless you're writing monster methods with more than 16 parameters, when would you ever need anything else? Well, there are some cases that cannot be expressed with generic type arguments. For example, if you need a delegate that can work with `ref`, `in`, or `out` parameters, you can't just write, say, `Func<bool, string, out int>`. This is because there is no such type as `out int` in .NET. When we use the `out` keyword in a method declaration, that's a statement about exactly how the argument works, so an `out int` parameter's type is still `int`. Generic type arguments only get to specify a type and cannot fully convey the distinction between `in`, `out`, and `ref` parameters.² So in these cases, you'll have to write a matching delegate type.

Another reason to define a custom delegate type is that you cannot use a `ref struct` as a generic type argument. ([Chapter 18](#) discusses these types.) So if you try to instantiate the generic `Action<T>` type with the `ref struct` type `Span<int>`, by writing `Action<Span<int>>`, you will get a compiler error. This limitation exists because `ref struct` types can only be used in certain scenarios (they must always live on the stack), and there's no way to determine whether any particular generic type or method uses its type arguments only in the ways that are allowed. (You could imagine a new kind of type argument constraint that expressed

this, but at the time of writing this, no such constraint exists.) So if you want a delegate type that can refer to a method that takes a `ref struct` argument, it needs to be a dedicated, nongeneric delegate.

NOTE

If you're relying on the compiler to determine a delegate expression's natural type (e.g., you write `var m = SomeMethod;`), these cases in which the `Func` and `Action` delegates cannot be used are the cases in which the compiler will generate a delegate type for you.

None of these restrictions explains why the runtime libraries define a separate `Predicate<T>` delegate type. `Func<T, bool>` would work perfectly well. Sometimes this kind of specialized delegate type exists as an accident of history: many delegate types have been around since before these general-purpose `Action` and `Func` types were added. But that's not the only reason—new delegate types continue to be added even now. The main reason is that sometimes it's useful to define a specialized delegate type to indicate particular semantics.

If you have a `Func<T, bool>`, all you know is that you've got a method that takes a `T` and returns a `bool`. But with a `Predicate<T>`, there's an implied meaning: it makes a decision about that `T` instance and returns `true` or `false` accordingly; not all methods that take a single argument and return a `bool` necessarily fit that pattern. By providing a `Predicate<T>`, you're not just saying that you have a method with a particular signature; you're saying you have a method that serves a particular purpose. For example, `HashSet<T>` (described in [Chapter 5](#)) has an `Add` method that takes a single argument and returns a `bool`, so it matches the signature of `Predicate<T>` but not the semantics. `Add`'s main job is to perform an action with side effects, returning some information about what it did, whereas predicates just tell you something about a value or object.

The runtime libraries define many delegate types, most of them even more specialized than `Predicate<T>`. For example, the `System.IO` namespace and its descendants define several that relate to specific events, such as `SerialPinChangedEventHandler`, which is used only

when you're working with old-fashioned serial ports such as the once-ubiquitous RS232 interface.

Type Compatibility

Delegate types do not derive from one another. Any delegate type you define in C# will derive directly from `MulticastDelegate`, as do all of the delegate types in the runtime libraries. However, the type system supports certain implicit reference conversions for generic delegate types through covariance and contravariance. The rules are very similar to those for interfaces. As the `in` keyword in [Example 9-3](#) showed, the type parameter `T` in `Predicate<T>` is contravariant, which means that if an implicit reference conversion exists between two types, `A` and `B`, an implicit reference conversion also exists between the types `Predicate<B>` and `Predicate<A>`. [Example 9-16](#) shows an implicit conversion that this enables.

Example 9-16. Delegate contravariance

```
public static bool IsLongString(object o)
{
    return o is string s && s.Length > 20;
}

static void Main(string[] args)
{
    Predicate<object> po = IsLongString;
    Predicate<string> ps = po;
    Console.WriteLine(ps("Too short"));
}
```

The `Main` method first creates a `Predicate<object>` referring to the `IsLongString` method. Any target method for this predicate type is capable of inspecting any `object` of any kind; thus, it's clearly able to meet the needs of code that requires a predicate capable of inspecting strings. It therefore makes sense that the implicit conversion to `Predicate<string>` should succeed—which it does, thanks to contravariance. Covariance also works in the same way as it does with interfaces, so it would typically be associated with a delegate's return type. (We denote

covariant type parameters with the `out` keyword.) All of the built-in `Func` delegate types have a covariant type parameter representing the function's return type called `TResult`. The type parameters for the function's parameters are all contravariant, as are all of the type parameters for the `Action` delegate types.

NOTE

The variance-based delegate conversions are implicit reference conversions. This means that when you convert the reference, the result still refers to the same delegate instance. (All implicit *reference* conversions have this characteristic, but not all implicit conversions work this way. Implicit numeric conversions create a new instance of the target type; implicit boxing conversions create a new box on the heap.) So in [Example 9-16](#), `po` and `ps` refer to the same delegate on the heap. This is subtly different from assigning `IsLongString` into both variables—that would create two delegates of different types.

You might also expect delegates that look the same to be compatible. For example, a `Predicate<int>` can refer to any method that a `Func<int, bool>` can use, and vice versa, so you might expect an implicit conversion to exist between these two types. You might be further encouraged by the “Delegate compatibility” section in the C# specification, which says that delegates with identical parameter lists and return types are compatible. (In fact, it goes further, saying that certain differences are allowed. For example, I mentioned earlier that argument types may be different as long as certain implicit reference conversions are available.) However, if you try the code in [Example 9-17](#), it won't work.

Example 9-17. Illegal delegate conversion

```
Predicate<string> pred = IsLongString;  
Func<string, bool> f = pred; // Will fail with compiler error
```

Adding an explicit cast doesn't work either—it removes the compiler error, but you just get a runtime error instead. The CTS considers these to be incompatible types, so a variable declared with one delegate type cannot hold a reference to a different delegate type even if their method signatures are compatible (except for when the two delegate types in ques-

tion are based on the same generic delegate type and are compatible thanks to covariance or contravariance). This is not the scenario for which C#'s delegate compatibility rules are designed—they are mainly used to determine whether a particular method can be the target for a particular delegate type.

The lack of type compatibility between “compatible” delegate types may seem odd, but structurally identical delegate types don’t necessarily have the same semantics, as we’ve already seen with `Predicate<T>` and `Func<T, bool>`. If you find yourself needing to perform this sort of conversion, it may be a sign that something is not quite right in your code’s design.³

Behind the Syntax

Although it takes just a single line of code to define a delegate type (as [Example 9-3](#) showed), the compiler turns this into a type that defines three methods and a constructor. Of course, the type also inherits members from its base classes. All delegates derive from `MulticastDelegate`, although all of the interesting instance members come from its base class, `Delegate`. (`Delegate` inherits from `object`, so delegates all have the ubiquitous `object` methods too.) Even `GetInvocationList`, clearly a multicast-oriented feature, is defined by the `Delegate` base class.

NOTE

The split between `Delegate` and `MulticastDelegate` is the meaningless and arbitrary result of a historical accident. The original plan was to support both multicast and unicast delegates, but toward the end of the prerelease period for .NET 1.0 this distinction was dropped, and now all delegate types support multicast instances. This happened sufficiently late in the day that Microsoft felt it was too risky to merge the two base types into one, so the split remained even though it serves no purpose.

I’ve already mentioned a couple of the public instance members that `Delegate` defines: the `DynamicInvoke` and `GetInvocationList` methods. There are two more. The `Method` property returns the `MethodInfo` representing the target method. ([Chapter 13](#) describes the `MethodInfo`

type.) The `Target` property returns the object that will be passed as the implicit `this` argument of the target method; if the delegate refers to a static method, `Target` will return `null`. [Example 9-18](#) shows the signatures of the compiler-generated constructor and methods for a delegate type. The details vary from one type to the next; these are the generated members in the `Predicate<T>` type.

Example 9-18. The members of a delegate type

```
public Predicate(object target, IntPtr method);

public bool Invoke(T obj);

public IAsyncResult BeginInvoke(T obj, AsyncCallback callback, object state);
public bool EndInvoke(IAsyncResult result);
```

Any delegate type you define will have four similar members. After compilation, none of them will have bodies yet. The compiler generates only their declarations, because the CLR supplies their implementations at runtime.

The constructor takes the target object (which is `null` for static methods) and an `IntPtr` identifying the method.⁴ Notice that this is not the `MethodInfo` returned by the `Method` property. Instead, this is a *function token*, an opaque binary identifier for the target method. The CLR can provide binary metadata tokens for all members and types, but there's no C# syntax for working with them, so we don't normally see them. When you construct a new instance of a delegate type, the compiler automatically generates IL that fetches the function token. The reason delegates use tokens internally is that they can be more efficient than working with reflection API types such as `MethodInfo`.

The `Invoke` method is the one that calls the delegate's target method (or methods). You can use this explicitly from C#, as [Example 9-19](#) shows. It is almost identical to [Example 9-13](#), the only difference being that the delegate variable is followed by `.Invoke`. This generates exactly the same code as [Example 9-13](#), so whether you write `Invoke` or just use the syntax that treats delegate identifiers as though they were method names is a matter of style. As a former C++ developer, I've always felt at home with

the [Example 9-13](#) syntax, because it's similar to using function pointers in that language, but there's an argument that writing `Invoke` explicitly makes it easier to see that the code is using a delegate.

Example 9-19. Using `Invoke` explicitly

```
public static void CallMeRightBack(Predicate<int> userCallback)
{
    bool result = userCallback.Invoke(42);
    Console.WriteLine(result);
}
```

One benefit of this explicit form is that you can use the null-conditional operator to handle the case where the delegate variable is null.

[Example 9-20](#) uses this to attempt invocation only when a non-null argument is supplied.

Example 9-20. Using `Invoke` with the null-conditional operator

```
public static void CallMeMaybe(Action<int>? userCallback)
{
    userCallback?.Invoke(42);
}
```

The `Invoke` method is the home for a delegate type's method signature. When you define a delegate type, this is where the return type and parameter list you specify end up. When the compiler needs to check whether a particular method is compatible with a delegate type (e.g., when you create a new delegate of that type), the compiler compares the `Invoke` method with the method you've supplied.

As [Example 9-18](#) shows, all delegate types also have `BeginInvoke` and `EndInvoke` methods. These used to provide a way to use the thread pool, but they are deprecated and don't work on the current version of .NET. (You'll get a `PlatformNotSupportedException` if you call either method.) They still work on .NET Framework, but they are obsolete. You should ignore these outdated methods and use the techniques described in [Chapter 16](#) instead. The main reason these methods used to be popular is that they provided an easy way to pass a set of values from one thread to

another—you could just pass whatever you needed as the arguments for the delegate. However, C# now has a much better way to solve the problem: anonymous functions.

Anonymous Functions

C# lets you create delegates without needing to define a separate method explicitly. You can write a special kind of expression whose value is a method. You could think of them as *method expressions* or *function expressions*, but the official name is *anonymous functions*. Expressions can be passed directly as arguments or assigned directly into variables, so the methods these expressions produce don't have names. (At least, not in C#. The runtime requires all methods to have names, so C# generates hidden names for these things, but from a C# language perspective, they are anonymous.)

For simple methods, the ability to write them inline as expressions can remove a lot of clutter. And as we'll see in [“Captured Variables”](#), the compiler exploits the fact that delegates are more than just a reference to a method to provide anonymous functions with access to any variables that were in scope in the containing method at the point at which the anonymous function appears.

For historical reasons, C# provides two ways to define an anonymous function. The older way involves the `delegate` keyword and is shown in [Example 9-21](#). This form is known as an *anonymous method*.⁵ I've put each argument for `FindIndex` on a separate line to make the anonymous function (the second argument) stand out, but C# does not require this.

Example 9-21. Anonymous method syntax

```
public static int GetIndexOffirstNonEmptyBin(int[] bins)
{
    return Array.FindIndex(
        bins,
        delegate (int value) { return value > 0; }
    );
}
```

In some ways, this resembles the normal syntax for defining methods. The parameter list appears in parentheses and is followed by a block containing the body of the method (which can contain as much code as you like, by the way, and is free to contain nested blocks, local variables, loops, and anything else you can put in a normal method). But instead of a method name, we just have the keyword `delegate`. The compiler infers the return type. In this case, the `FindIndex` method's signature declares the second parameter to be a `Predicate<T>`, which tells the compiler that the return type has to be `bool`.

In fact, the compiler knows more than just the return type. I've passed `FindIndex` an `int[]` array, so the compiler will deduce that the type argument `T` is `int`, making the second argument a `Predicate<int>`. This means that in [Example 9-21](#), I had to supply information—the type of the delegate's parameter—that the compiler already knew. A later version of C# introduced a more compact anonymous function syntax that takes better advantage of what the compiler can deduce, shown in [Example 9-22](#).

Example 9-22. Lambda syntax

```
public static int GetIndexOffirstNonEmptyBin(int[] bins)
{
    return Array.FindIndex(
        bins,
        value => value > 0
    );
}
```

This form of anonymous function is called a *lambda expression*, and it is named after a branch of mathematics that is the foundation of a function-based model for computation. There is no particular significance to the choice of the Greek letter lambda (λ). It was the accidental result of the limitations of 1930s printing technology. The inventor of lambda calculus, Alonzo Church, originally wanted a different notation, but when he published his first paper on the subject, the typesetting machine operator decided to print λ instead, because that was the closest approximation to Church's notation that the machine could produce. Despite these inauspicious origins, this arbitrarily chosen term has become ubiquitous. LISP, an early and influential programming language, used the name *lambda*

for expressions that are functions, and since then, many languages have followed suit, including C#.

[Example 9-22](#) is exactly equivalent to [Example 9-21](#); I've just been able to leave various things out. The `=>` token unambiguously marks this out as being a lambda, so the compiler does not need that cumbersome and ugly `delegate` keyword just to recognize this as an anonymous function. The compiler knows from the surrounding context that the method has to take an `int`, so there's no need to specify the parameter's type; I just provided the parameter's name: `value`. For simple methods that consist of just a single expression, the lambda syntax lets you omit the block and the `return` statement. This all makes for very compact lambdas, but in some cases, you might not want to omit quite so much, so as [Example 9-23](#) shows, there are various optional features. Every lambda in this example is equivalent.

Example 9-23. Lambda variations

```
Predicate<int> p1 = value => value > 0;
Predicate<int> p2 = (value) => value > 0;
Predicate<int> p3 = (int value) => value > 0;
Predicate<int> p4 = value => { return value > 0; };
Predicate<int> p5 = (value) => { return value > 0; };
Predicate<int> p6 = (int value) => { return value > 0; };
Predicate<int> p7 = bool (value) => value > 0;
Predicate<int> p8 = bool (int value) => value > 0;
Predicate<int> p9 = bool (value) => { return value > 0; };
Predicate<int> pA = bool (int value) => { return value > 0; };
```

The first variation is that you can put parentheses around the parameter. This is optional with a single parameter, but it is mandatory for multiparameter lambdas. You can also be explicit about the parameters' types (in which case you will also need parentheses, even if there's only one parameter). And, if you like, you can use a block instead of a single expression, at which point you also have to use the `return` keyword if the lambda returns a value. The normal reason for using a block would be if you wanted to write multiple statements inside the method. The final four lines show that you can specify the return type explicitly, although that's only allowed when the parameter list is in parentheses.

You may be wondering why there are quite so many different forms—why not have just one syntax and be done with it? Although the final line of [Example 9-23](#) shows the most general form, it's also a lot more cluttered than the first line. Since one of the goals of lambdas is to provide a more concise alternative to anonymous methods, C# supports these shorter forms where they can be used without ambiguity.

You can also write a lambda that takes no arguments. As [Example 9-24](#) shows, we just put an empty pair of parentheses in front of the `=>` token. (And, as this example also shows, lambdas that use the greater than or equals operator, `>=`, can look a bit odd due to the meaningless similarity between the `=>` and `>=` tokens.)

Example 9-24. A zero-argument lambda

```
Func<bool> isAfternoon = () => DateTime.Now.Hour >= 12;
```

The flexible and compact syntax means that lambdas have all but displaced the older anonymous method syntax. However, the older syntax offers one advantage: it allows you to omit the parameter list entirely. In some situations where you provide a callback, you need to know only that whatever you were waiting for has now happened. This is particularly common when using the standard event pattern described later in this chapter, because that requires event handlers to accept arguments even in situations where they serve no purpose. For example, when a button is clicked, there's not much else to say beyond the fact that it was clicked, and yet all of the button types in .NET's various UI frameworks pass two arguments to the event handler. [Example 9-25](#) successfully ignores this by using an anonymous method that omits the parameter list.

Example 9-25. Ignoring arguments in an anonymous method

```
EventHandler clickHandler = delegate { Debug.WriteLine("Clicked!"); };
```

`EventHandler` is a delegate type that requires its target methods to take two arguments, of type `object` and `EventArgs`. If our handler needed access to either, we could, of course, add a parameter list, but the anonymous method syntax lets us leave it out if we want. You cannot do this

with a lambda. That said, lambdas offer a succinct way to ignore arguments, which [Example 9-26](#) illustrates.

Example 9-26. A lambda discarding its arguments

```
EventHandler clickHandler = (_, _) => Debug.WriteLine("Clicked!");
```

This has exactly the same effect as [Example 9-25](#) but using the lambda syntax. I've provided an argument list in parentheses, but because I don't want to use either argument, I've put an underscore in each position. This denotes a *discard*. You have seen the `_` character in patterns in early chapters, and it's broadly similar in meaning here: it indicates that we know there's a value available; it's just that we don't care what it is and don't intend to use it.

TIP

Before C# introduced support for this discard syntax, people would often use a similar-looking convention. The underscore symbol is a valid identifier, so for single-argument lambdas, nothing stops you from defining an argument named `_` and choosing not to refer to it. It got weird with multiple arguments because you can't use the same name for two arguments, meaning [Example 9-26](#) would not compile on older versions of C#. To work around this, people just used multiple underscores, so you might see a lambda starting `(_, __, ___) =>`. Thankfully, C# now allows us to use a single `_` throughout.

Lambdas and Default Arguments

You saw earlier that a delegate type can define default argument values for some or all of its parameters. Prior to C# 12.0, you had to define your own delegate type explicitly to do this, but you can now also do it with the lambda syntax. [Example 9-27](#) shows a delegate specifying a default argument of 10 for its only parameter.

Example 9-27. A lambda specifying a default argument value

```
var withDefault = (int x = 10) => x * 2;
```

I've used `var` here to demonstrate what happens when you get the compiler to infer the delegate type for a lambda with default arguments. Normally, a lambda like this that takes a single `int` argument and returns an `int` result would become a `Func<int, int>`, but the presence of the `= 10` changes that. `Func<int, int>` represents a method with a single parameter that does not have a default argument. (Generic type parameters are always types, so although you could imagine some syntax like `ImpossibleType<int, 10, int>` it's not possible to write a generic type that works this way—you can't pass a value as a generic type argument.) Since the runtime libraries do not define any type that can be used as the natural type for `withDefault` in [Example 9-27](#), the compiler will generate an anonymous type similar to the one shown in [Example 9-28](#), which shows how we might have achieved the same end result before C# 12.0.

In fact, the compiler puts this default argument value in two places. It becomes part of the generated generic type, but it also becomes part of the hidden method the compiler generates to hold the code for the lambda. You can see this in the following example, [Example 9-28](#)—that `x = 10` shows up not only in the `WithDefaultDelegate` type but also in the `WithDefaultMethod`.

Example 9-28. The approximate effect of a lambda specifying a default argument value

```
public delegate int WithDefaultDelegate(int x = 10);

public static class EffectOfLambdaWithDefaultArg
{
    public static void UseLambda()
    {
        WithDefaultDelegate withDefault = WithDefaultMethod;
        Console.WriteLine($"Default arg: {withDefault()}");
        Console.WriteLine($"Supplied arg: {withDefault(42)}");
    }

    private static int WithDefaultMethod(int x = 10)
    {
        return x * 2;
    }
}
```

It's important that the compiler specifies the default value not just on the delegate type but also on the method itself, because it enables one of the main intended scenarios for this new C# 12.0 feature. Some frameworks, most notably the ASP.NET Core web framework, will detect when methods have default argument values, and modify their behavior accordingly. [Example 9-29](#) shows a simple but complete web application that accepts requests just on the root URL (e.g., `https://localhost/`).

Example 9-29. Using a lambda with a default argument in ASP.NET Core

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

app.MapGet("/", (string name = "World") => $"Hello {name}!");

app.Run();
```

This supplies a lambda to `MapGet` to provide the code to run for any request for that URL. It defines a single argument, `name`, and if a client supplies a value for that in a query string, e.g., `https://localhost/?name=Ian`, the value will be passed to this code. But since the lambda has specified a default value, ASP.NET Core knows that this is optional, and it will pass in that default value of `"World"` if the client request did not supply something else.

ASP.NET Core looks at the target method (and not the delegate type) to determine whether there are any default arguments. It works that way because ASP.NET Core supported default values in earlier versions, before C# 12.0 made it possible for lambdas to specify default arguments. You used to have to write an explicit method as [Example 9-30](#) does to have somewhere to put the default argument value. As it happens, with this example C# 12.0 will generate a delegate type that includes the default argument, but since earlier compilers didn't do that, ASP.NET Core had to look at the target method itself. It continues to do that in the current version to ensure compatibility.

Example 9-30. Specifying a default argument in ASP.NET Core without

a lambda

```
static string HandleRootUrl(string name = "World")
{
    return $"Hello {name}!";
}
app.MapGet("/", HandleRootUrl);
```

Since the default argument goes in two places—the delegate type and the target method—it’s possible to create situations in which these are mismatched. [Example 9-31](#) does this, first by assigning two lambdas with different default arguments into the same variable, and then by assigning lambdas into variables whose explicit types do not match the default arguments supplied.

Example 9-31. Mismatched default arguments

```
// The compiler generates a delegate type with a default argument of 10.
var withDefault = (int x = 10) => x * 2;

// Warning because this has a different default value (20) than withDefault's
// compiler-generated delegate type.
withDefault = (int x = 20) => x + 2;

// The WithDefaultDelegate delegate defined in a previous example also
// specifies a default argument of 10 so this also generates a warning.
WithDefaultDelegate c = (int x = 20) => x - 1;

// Also warns, because the delegate type does not specify a default argument,
// but the lambda does.
Func<int, int> f = (int x = 20) => x + 3;
```

For all but the first assignment, the compiler generates warnings telling you that the target method’s default argument value does not match the default value specified by the delegate type. (On the final line, the delegate type does not specify a default argument at all, but this too produces a warning when you use it with a lambda that does have a default argument.) The effect of these mismatches depends on how the resulting delegate is used. You’ve already seen that ASP.NET Core ignores any default in the delegate type, and looks only at the target method (to retain compatibility with pre-C# 12.0 behavior). But code that just invokes a delegate in

the normal way will use the default value from the delegate type, ignoring any defaults specified by the target method itself. In practice it's best to avoid getting into a situation where you need to know which of the two defaults will be used, which is why C# warns you if you create a situation in which they will be different.

Captured Variables

While anonymous functions often take up much less space in your source code than a full, normal method, they're not just about conciseness. The C# compiler uses a delegate's ability to refer not just to a method but also to some additional context to provide an extremely useful feature: it can make variables from the containing method available to the anonymous function. [Example 9-32](#) shows a method that returns a `Predicate<int>`. It creates this with a lambda that uses an argument from the containing method.

Example 9-32. Using a variable from the containing method

```
public static Predicate<int> IsGreaterThan(int threshold)
{
    return value => value > threshold;
}
```

This provides the same functionality as the `ThresholdComparer` class in [Example 9-9](#), but instead of having to write an entire class, we need only a single, simple method. We can make this even more compact by using an expression-bodied method, as [Example 9-33](#) shows. (This might be a bit *too* concise—two different uses of `=>` in close proximity to `>` won't win any prizes for readability.)

Example 9-33. Using a variable from the containing method (expression-bodied)

```
public static Predicate<int> IsGreaterThan(int threshold) =>
    value => value > threshold;
```

In either form, the code is almost deceptively simple, so it's worth looking closely at what it does. The `IsGreaterThan` method returns a delegate in-

stance. That delegate's target method performs a simple comparison—it evaluates the `value > threshold` expression and returns the result. The `value` variable in that expression is just the delegate's argument—the `int` passed by whichever code invokes the `Predicate<int>` that `IsGreaterThan` returns. The second line of [Example 9-34](#) invokes that code, passing in 200 as the argument for `value`.

Example 9-34. Where the `value` argument comes from

```
Predicate<int> greaterThanTen = IsGreaterThan(10);  
bool result = greaterThanTen(200);
```

The `threshold` variable in the expression is trickier. This is not an argument to the anonymous function. It's the argument of `IsGreaterThan`, and [Example 9-34](#) passes a value of 10 as the `threshold` argument. However, `IsGreaterThan` has to return before we can invoke the delegate it returns. Since the method for which that `threshold` variable was an argument has already returned, you might think that the variable would no longer be available by the time we invoke the delegate. In fact, it's fine, because the compiler does some work on our behalf. If an anonymous function uses local variables that were declared by the containing method, or if it uses that method's parameters, the compiler generates a class to hold those variables so that they can outlive the method that created them. The compiler generates code in the containing method to create an instance of this class. (Remember, each invocation of a block gets its own set of local variables, so if any locals get pushed into an object to extend their lifetime, a new object will be required for each invocation.) This is one of the reasons why the popular myth that says local variables of value type always live on the stack is not true—in this case, the compiler copies the incoming `threshold` argument's value to a field of an object on the heap, and code that uses the `threshold` variable ends up using that field instead. [Example 9-35](#) shows the generated code that the compiler produces for the anonymous function in [Example 9-32](#).

Example 9-35. Code generated for an anonymous function

```
[CompilerGenerated]  
private sealed class <>c__DisplayClass0_0  
{
```



```

    public int threshold;

    public bool <IsGreaterThan>b__0(int value)
    {
        return (value > this.threshold);
    }
}

```

The class and method names all begin with characters that are illegal in C# identifiers, to ensure that this compiler-generated code cannot clash with anything we write—this is technically an *unspeakable name*. (The exact names are not fixed, by the way—you may find they are slightly different if you try this.) This generated code bears a striking resemblance to the `ThresholdComparer` class from [Example 9-9](#), which is unsurprising, because the goal is the same: the delegate needs some method that it can refer to, and that method’s behavior depends on a value that is not fixed. Anonymous functions are not a feature of the runtime’s type system, so the compiler has to generate a class to provide this kind of behavior on top of the CLR’s basic delegate functionality.

Once you know that this is what’s really happening when you write an anonymous function, it follows naturally that the inner method is able not just to read the variable but also to modify it. This variable is just a field in an object that two methods—the anonymous function and the containing method—have access to. [Example 9-36](#) uses this to maintain a count that is updated from an anonymous function.

Example 9-36. Modifying a captured variable

```

static void Calculate(int[] nums)
{
    int zeroEntryCount = 0;
    int[] nonZeroNums = Array.FindAll(
        nums,
        v =>
        {
            if (v == 0)
            {
                zeroEntryCount += 1;
                return false;
            }
            else

```

```

        {
            return true;
        }
    });
    Console.WriteLine($"Number of zero entries: {zeroEntryCount}");
    Console.WriteLine($"First nonzero entry: {nonZeroNums[0]}");
}

```

Everything in scope for the containing method is also in scope for anonymous functions. If the containing method is an instance method, this includes any instance members of the type, so your anonymous function could access fields, properties, and methods. (The compiler supports this by adding a field to the generated class to hold a copy of the `this` reference.) The compiler puts only what it needs to in generated classes of the kind shown in [Example 9-35](#), and if you don't use variables or instance members from the containing scope, it might be able to generate a static method.

The `FindAll` method in the preceding examples does not hold on to the delegate after it returns—any callbacks will happen while `FindAll` runs. Not everything works that way, though. Some APIs perform asynchronous work and will call you back at some point in the future, by which time the containing method may have returned. This means that any variables captured by the anonymous function will live longer than the containing method. In general, this is fine, because all of the captured variables live in an object on the heap, so it's not as though the anonymous function is relying on a stack frame that is no longer present. The one thing you need to be careful of, though, is explicitly releasing resources before callbacks have finished. [Example 9-37](#) shows an easy mistake to make. This uses an asynchronous, callback-based API to download the resource at a particular URL via HTTP. (This calls the `ContinueWith` method on the `Task<Stream>` returned by `HttpClient.GetStreamAsync`, passing a delegate that will be invoked once the HTTP response comes back. `ContinueWith` is part of the Task Parallel Library described in [Chapter 16](#).)

Example 9-37. Premature disposal

```

HttpClient http = GetHttpClient();
using (FileStream file = File.OpenWrite(@"c:\temp\page.txt"))

```

```
{  
    http.GetStreamAsync("https://endjin.com/")  
        .ContinueWith((Task<Stream> t) => t.Result.CopyToAsync(file));  
} // Will probably dispose FileStream before callback runs
```

The `using` statement in this example will dispose the `FileStream` as soon as execution reaches the point at which the `file` variable goes out of scope in the outer method. The problem is that this `file` variable is also used in an anonymous function, which will in all likelihood run after the thread executing that outer method has left that `using` statement's block. The compiler has no understanding of when the inner block will run—it doesn't know whether that's a synchronous callback like `Array.FindAll` uses or an asynchronous one. So it cannot do anything special here—it just calls `Dispose` at the end of the block, as that's what our code told it to do.

NOTE

The asynchronous language features discussed in [Chapter 17](#) can help avoid this sort of problem. When you use those to consume APIs that present this kind of `Task`-based pattern, the compiler can then know exactly how long things remain in scope. This enables the compiler to generate continuation callbacks for you, and as part of this, it can arrange for a `using` statement to call `Dispose` at the correct moment.

In performance-critical code, you may need to bear the costs of anonymous functions in mind. If the anonymous function uses variables from the outer scope, then in addition to the delegate object that you create to refer to the anonymous function, you may be creating an additional one: an instance of the generated class to hold shared local variables. The compiler will reuse these variable holders when it can—if one method contains two anonymous functions, they may be able to share an object, for example. Even with this sort of optimization, you're still creating additional objects, increasing the pressure on the GC. (And in some cases you can end up creating this object even if you never hit the code path that creates the delegate.) It's not particularly expensive—these are typically small objects—but if you're up against a particularly oppressive performance problem, you might be able to eke out some small improvements

by writing things in a more long-winded fashion in order to reduce the number of object allocations.

NOTE

Local functions do not always incur this same overhead. When a local function uses its outer method's variables, it does not extend their lifetime. The compiler therefore doesn't need to create an object on the heap to hold the shared variables. It still creates a type to hold all the shared variables, but it defines this as a `struct` that it passes by reference as a hidden `in` argument, avoiding the need for a heap block. (If you create a delegate that refers to a local function, it can no longer use this optimization, and it reverts to the same strategy it uses for anonymous functions, putting shared variables in an object on the heap.)

More subtly, using an outer scope's local variables in an anonymous function will extend the liveness of those variables, which may mean the GC will take longer to detect when objects those variables refer to are no longer in use. As you may recall from [Chapter 7](#), the CLR analyzes your code to work out when variables are in use so that it can free objects without waiting for the variables that refer to them to go out of scope. This enables the memory used by some objects to be reclaimed significantly earlier, particularly in methods that take a long time to complete. But liveness analysis applies only to conventional local variables. It cannot be applied for variables that are used in an anonymous function, because the compiler transforms those variables into fields. (From the CLR's perspective, they are not local variables at all.) Since C# typically puts all of these transformed variables for a particular scope into a single object, you will find that none of the objects these variables refer to can be reclaimed until the method completes and the object containing the variables becomes unreachable itself. This can mean that in some cases there may be a measurable benefit to setting a local variable to `null` when you're done with it, enabling that particular object's memory to be reclaimed at the next GC. (Normally, that would be bad advice, and even with anonymous functions it might not have a useful effect in practice. You should only do this if performance testing demonstrates a clear advantage. But it's worth investigating in cases where you're seeing GC-related performance problems and you make heavy use of long-running anonymous functions.)

You can easily avoid these potential performance downsides in anonymous functions: just don't use captured variables. If an anonymous function never tries to use anything from its containing scope, the C# compiler won't engage the corresponding mechanisms, completely avoiding all the overhead. You can tell the compiler that you are intending to avoid capturing variables by annotating it with the `static` keyword, as [Example 9-38](#) shows. Just as an ordinary `static` method does not have implicit access to an instance of its defining type, a `static` anonymous function has no access to its containing scope. This use of `static` doesn't change how code is generated—any anonymous function that does not rely on capture will avoid all capture-related overheads, regardless of whether it was marked as `static`. This just asks the compiler to report errors if you inadvertently attempt to use variables from the function's containing scope.

Example 9-38. Opting out of variable capture with `static`

```
public static Predicate<int> IsGreaterThan10() => static value => value > 10;
```

Variable capture can also occasionally lead to bugs, particularly due to a subtle scope-related issue with `for` loops. (`foreach` loops don't have this problem.) [Example 9-39](#) runs into this.

Example 9-39. Problematic variable capture in a `for` loop

```
public static void Caught()
{
    var greaterThanN = new Predicate<int>[10];
    for (int i = 0; i < greaterThanN.Length; ++i)
    {
        greaterThanN[i] = value => value > i; // Bad use of i
    }

    Console.WriteLine(greaterThanN[5](20));
    Console.WriteLine(greaterThanN[5](6));
}
```

This example initializes an array of `Predicate<int>` delegates, where each delegate tests whether the value is greater than some number. (You wouldn't have to use arrays to see the problem I'm about to describe, by

the way. Your loop might instead pass the delegates it creates into one of the mechanisms described in [Chapter 16](#) that enable parallel processing by running the code on multiple threads. But arrays make it easier to show the problem.) Specifically, it compares the value with `i`, the loop counter that decides where in the array each delegate goes, so you might expect the element at index 5 to refer to a method that compares its argument with 5. If that were so, this code would show `True` twice. In fact, it displays `True` and then `False`. It turns out that [Example 9-39](#) produces an array of delegates where every single element compares its argument with 10.

This usually surprises people when they encounter it. With hindsight, it's easy enough to see why this happens when you know how the C# compiler enables an anonymous function to use variables from its containing scope. The `for` loop declares the `i` variable, and because it is used not only by the containing `Caught` method but also by each delegate the loop creates, the compiler will generate a class similar to the one in [Example 9-35](#), and the variable will live in a field of that class. Since the variable comes into scope when the loop starts, and remains in scope for the duration of the loop, the compiler will create one instance of that generated class, and it will be shared by all of the delegates. So, as the loop increments `i`, this modifies the behavior of all of the delegates, because they all use that same `i` variable.

Fundamentally, the problem is that there's only one `i` variable here. You can fix the code by introducing a new variable inside the loop. [Example 9-40](#) copies the value of `i` into another local variable, `current`, which does not come into scope until an iteration is under way, and goes out of scope at the end of each iteration. So, although there is only one `i` variable, which lasts for as long as the loop runs, we get what is effectively a new `current` variable each time around the loop. Because each delegate gets its own distinct `current` variable, this modification means that each delegate in the array compares its argument with a different value—the value that the loop counter had for that particular iteration.

Example 9-40. Modifying a loop to capture the current value

```
for (int i = 0; i < greaterThanN.Length; ++i)
```

```

{
    int current = i;
    greaterThanN[i] = value => value > current;
}

```

The compiler still generates a class similar to the one in [Example 9-35](#) to hold the `current` variable that's shared by the inline and containing methods, but this time, it will create a new instance of that class each time around the loop in order to give each anonymous function a different instance of that variable. (When you use a `foreach` loop, the scoping rules are a little different: its iteration variable's scope is per iteration, meaning that it's logically a different instance of the variable each time around the loop, so there would be no need to add an extra variable inside the loop as we have to with `for`.)

You may be wondering what would happen if you wrote an anonymous function that used variables at multiple scopes. [Example 9-41](#) declares a variable called `offset` before the loop, and the lambda uses both that and a variable whose scope lasts for only one iteration.

Example 9-41. Capturing variables at different scopes

```

int offset = 10;
for (int i = 0; i < greaterThanN.Length; ++i)
{
    int current = i;
    greaterThanN[i] = value => value > (current + offset);
}

```

In that case, the compiler would generate two classes, one to hold any per-iteration shared variables (`current`, in this example) and one to hold those whose scope spans the whole loop (`offset`, in this case). Each delegate's target object would contain inner scope variables, and that would contain a reference to the outer scope.

[Figure 9-1](#) shows roughly how this would work, although it has been simplified to show just the first five items. The `greaterThanN` variable contains a reference to an array. Each array element contains a reference to a delegate. Each delegate refers to the same method, but each one has a dif-

ferent target object, which is how each delegate can capture a different instance of the `current` variable. Each of these target objects refers to a single object containing the `offset` variable captured from the scope outside of the loop.

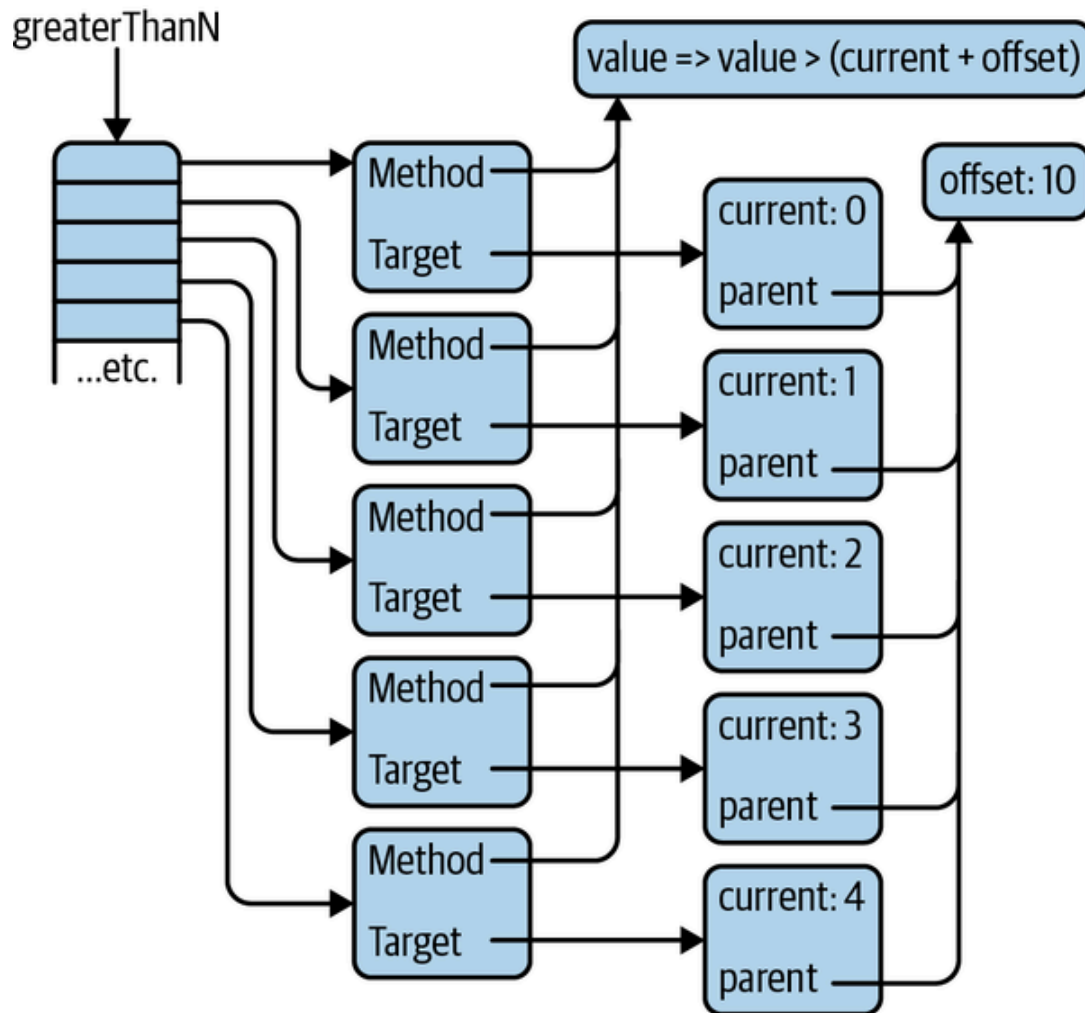


Figure 9-1. Delegates and captured scopes

Lambdas and Expression Trees

Lambdas have an additional trick up their sleeves beyond providing delegates. Some lambdas produce a data structure that represents code. This occurs when you use the lambda syntax in a context that requires an `Expression<T>`, where `T` is a delegate type. `Expression<T>` itself is not a delegate type; it is a special type in the runtime libraries (in the `System.Linq.Expressions` namespace) that triggers this alternative handling of lambdas in the compiler. [Example 9-42](#) uses this type.

Example 9-42. A lambda expression

```
Expression<Func<int, bool>> greaterThanZero = value => value > 0;
```

This example looks similar to some of the lambdas and delegates I’ve shown already in this chapter, but the compiler handles this very differently. It will not generate a method—there will be no compiled IL representing the lambda’s body. Instead, the compiler will produce code similar to that in [Example 9-43](#).

Example 9-43. What the compiler does with a lambda expression

```
ParameterExpression valueParam = Expression.Parameter(typeof(int), "value");  
ConstantExpression constantZero = Expression.Constant(0);  
BinaryExpression comparison = Expression.GreaterThan(valueParam, constantZero);  
Expression<Func<int, bool>> greaterThanZero =  
    Expression.Lambda<Func<int, bool>>(comparison, valueParam);
```

This code calls various factory functions provided by the `Expression` class to produce an object for each subexpression in the lambda. This starts with the `value` parameter and the constant value `0`. These are fed into an object representing the “greater than” comparison expression, which in turn becomes the body of an object representing the whole lambda expression.

The ability to produce an object model for an expression makes it possible to write an API where the behavior is controlled by the structure and content of an expression. For example, some data access APIs can take an expression similar to the ones produced by [Examples 9-42](#) and [9-43](#) and use it to generate part of a database query. I’ll be talking about C#’s integrated query features in [Chapter 10](#), but [Example 9-44](#) gives a flavor of how a lambda expression can be used as the basis of a query.

Example 9-44. Expressions and database queries

```
var expensiveProducts = dbContext.Products.Where(p => p.ListPrice > 3000);
```

This example happens to use a Microsoft library called the Entity Framework, but various other data access technologies support the same

approach. In this example, the `Where` method takes an argument of type `Expression<Func<Product, bool>>`.⁶ `Product` is a class that corresponds to an entity in the database, but the important part here is the use of `Expression<T>`. That means that the compiler will generate code that creates a tree of objects whose structure corresponds to that lambda expression. The `Where` method processes this expression tree, generating a SQL query that includes this clause: `WHERE [Extent1].[ListPrice] > cast(3000 as decimal(18))`. So, although I wrote my query as a C# expression, the work required to find matching objects will all happen on my database server.

Expression trees were added to C# to enable this sort of query handling as part of the set of features known collectively as *LINQ* (which is the subject of [Chapter 10](#)). However, as with most LINQ-related features, it's possible to use them for other things. For example, a popular .NET library used in automated testing called [Moq](#) exploits this. It creates fake implementations of interfaces for test purposes, and it uses lambda expressions to provide a simple API for configuring how those fakes should behave. [Example 9-45](#) uses Moq's `Mock<T>` class to create a fake implementation of .NET's `IEqualityComparer<string>` interface. The code calls the `Setup` method, which takes an expression indicating a specific invocation we'd like to define special handling for—in this case, if the fake's implementation of `IEqualityComparer<string>.Equals` is called with the arguments of `"Color"` and `"Colour"`, we'd like it to return `true`.

Example 9-45. Use of lambda expressions by the Moq library

```
var fakeComparer = new Mock<IEqualityComparer<string>>();
fakeComparer
    .Setup(c => c.Equals("Color", "Colour"))
    .Returns(true);
```

If that argument to `Setup` were just a delegate, there would be no way for Moq to inspect it. But because it's an expression tree, Moq is able to delve into it and find out what we've asked for.

WARNING

Unfortunately, expression trees are an area of C# that have lagged behind the rest of the language. They were introduced in C# 3.0, and various language features added since then, such as support for tuples and asynchronous expressions, can't be used in an expression tree because the object model has no way to represent them.

Events

Sometimes it is useful for objects to be able to provide notifications of when interesting things have happened—in a client-side UI framework, you will want to know when the user clicks one of your application's buttons, for example. Delegates provide the basic callback mechanism required for notifications, but there are many ways you could go about using them. Should the delegate be passed as a method argument, a constructor argument, or perhaps as a property? How should you support unsubscribing from notifications? The CTS formalizes the answers to these questions through a special kind of class member called an *event*, and C# has syntax for working with events. [Example 9-46](#) shows a class with one event member.

Example 9-46. A class with an event

```
public class Eventful
{
    public event Action<string>? Announcement;

    public void Announce(string message)
    {
        Announcement?.Invoke(message);
    }
}
```

As with all members, you can start with an accessibility specifier, and it will default to `private` if you leave that off. Next, the `event` keyword singles this out as an event. Then there's the event's type, which can be any delegate type. I've used `Action<string>`, although as you'll soon see,

this is an unorthodox choice. Finally, we put the member name, so this example defines an event called `Announcement`.

To handle an event, you must provide a delegate of the right type, and you must use the `+=` syntax to attach that delegate as the handler.

[Example 9-47](#) uses a lambda, but you can use any expression that produces, or is implicitly convertible to, a delegate of the type the event requires.

Example 9-47. Handling events

```
var source = new Eventful();
source.Announcement += m => Console.WriteLine($"Announcement: {m}");
```

As well as defining an event, [Example 9-46](#) also shows how to *raise* it—that is, how to invoke all the handlers that have been attached to the event. Its `Announce` uses the same syntax we would use if `Announcement` were a field containing a delegate that we wanted to invoke. In fact, as far as the code inside the class is concerned, that’s exactly what an event looks like—it appears to be a field. I’ve chosen to use the delegate’s `Invoke` member explicitly here instead of writing `Announcement(message)` because this lets me use the null-conditional operator (`?.`). This causes the compiler to generate code that invokes the delegate only if it is not null. Otherwise I would have had to write an `if` statement verifying that the field is not null before invoking it.

So why do we need a special member type if this looks just like a field? Well, it looks like a field only from inside the defining class. Code outside of the class cannot raise the event, so the code shown in [Example 9-48](#) will not compile.

Example 9-48. How not to raise an event

```
var source = new Eventful();
source.Announcement("Will this work?"); // No, this will not even compile
```

From the outside, the only things you can do to an event are to attach a handler using `+=` and to remove one using `-=`. The syntax for adding and removing event handlers is unusual in that it’s the only case in C# in

which you get to use `+=` and `-=` without the corresponding standalone `+` or `-` operators being available. The actions performed by `+=` and `-=` on events both turn out to be method calls in disguise. Just as properties are really pairs of methods with a special syntax, so are events. They are similar in concept to the code shown in [Example 9-49](#). (In fact, the real code includes some moderately complex lock-free, thread-safe code. I've not shown this because the multithreading obscures the basic intent.) This won't have quite the same effect, because the `event` keyword adds metadata to the type identifying the methods as being an event, so this is just for illustration.

Example 9-49. The approximate effect of declaring an event

```
private Action<string>? Announcement;

// Not the actual code.
// The real code is more complex, to tolerate concurrent calls.
public void add_Announcement(Action<string> handler)
{
    Announcement += handler;
}
public void remove_Announcement(Action<string> handler)
{
    Announcement -= handler;
}
```

Just as with properties, events exist mainly to offer a convenient, distinctive syntax and to make it easier for tools to know how to present the features that classes offer. Events are particularly important for UI elements. In most UI frameworks, the objects representing interactive elements can often raise a wide range of events, corresponding to various forms of input such as keyboard, mouse, or touch. There are also often events relating to behavior specific to a particular control, such as selecting a new item in a list. Because the CTS defines a standard idiom by which elements can expose events, visual UI designers, such as the ones built into Visual Studio, can display the available events and offer to generate handlers for you.

Standard Event Delegate Pattern

The event in [Example 9-46](#) is unusual in that it uses the `Action<T>` delegate type. This is perfectly legal, but in practice, you will rarely see that, because almost all events use delegate types that conform to a particular pattern. This pattern requires the delegate's method signature to have two parameters. The first parameter's type is `object`, and the second's type is either `EventArgs` or some type derived from `EventArgs`.

[Example 9-50](#) shows the `EventHandler` delegate type in the `System` namespace, which is the simplest and most widely used example of this pattern.

Example 9-50. The `EventHandler` delegate type

```
public delegate void EventHandler(object sender, EventArgs e);
```

The first parameter is usually called `sender`, because the event source passes a reference to itself for this argument. This means that if you attach a single delegate to multiple event sources, that handler can always know which source raised any particular notification.

The second parameter provides a place to put information specific to the event. For example, WPF UI elements define various events for handling mouse input that use more specialized delegate types, such as `MouseButtonEventHandler`, with signatures that specify a corresponding specialized event parameter that offers details about the event. For example, `MouseButtonEventArgs` defines a `GetPosition` method that tells you where the mouse was when the button was clicked, and it defines various other properties offering further detail, including `ClickCount` and `Timestamp`.

Whatever the specialized type of the second parameter may be, it will always derive from the base `EventArgs` type. That base type is not very interesting—it does not add members beyond the standard ones provided by `object`. However, it does make it possible to write a general-purpose method that can be attached to any event that uses this pattern. The rules for delegate compatibility mean that even if the delegate type specifies a second parameter of type `MouseButtonEventArgs`, a method whose second parameter is of type `EventArgs` is an acceptable target. This can oc-

asionally be useful for code generation or other infrastructure scenarios. However, the main benefit of the standard event pattern is simply one of familiarity—experienced C# developers generally expect events to work this way.

Custom Add and Remove Methods

Sometimes, you might not want to use the default event implementation generated by the C# compiler. For example, a class may define a large number of events, most of which will not be used on the majority of instances. UI frameworks often have this characteristic. A WPF UI can have thousands of elements, every one of which offers over 100 events, but you normally attach handlers only to a few of these elements, and even with these, you handle only a fraction of the events on offer. It is inefficient for every element to dedicate a field to every available event in this case.

Using the default field-based implementation for large numbers of rarely used events could add hundreds of bytes to the footprint of each element in a UI, which can have a discernible effect on performance. (In a typical WPF application, this could add up to a few hundred thousand bytes. That might not sound like much given modern computers' memory capacities, but it can put your code in a place where it is no longer able to make efficient use of the CPU's cache, causing a nosedive in application responsiveness. Even if the cache is several megabytes in size, the fastest parts of the cache are usually much smaller, and wasting a few hundred kilobytes in a critical data structure can make a world of difference to performance.)

Another reason you might want to eschew the default compiler-generated event implementation is that you may want more sophisticated semantics when raising events. For example, WPF supports *event bubbling*: if a UI element does not handle certain events, they will be offered to the parent element, then the parent's parent, and so on up the tree until a handler is found or it reaches the top. Although it would be possible to implement this sort of scheme with the standard event implementation C# supplies, much more efficient strategies are possible when event handlers are relatively sparse.

To support these scenarios, C# lets you provide your own add and remove methods for an event. It will look just like a normal event from the out-

side—anyone using your class will use the same `+=` and `-=` syntax to add and remove handlers—and it won't be possible to tell that it provides a custom implementation. [Example 9-51](#) shows a class with two such events, and it uses a single dictionary, shared across all instances of the class, to keep track of which events have been handled on which objects. The approach is extensible to larger numbers of events—the dictionary uses pairs of objects as the key, so each entry represents a particular (source, event) pair. (This is not production-quality code, by the way. It's not safe for multithreaded use, and it will also leak memory when a `ScarceEventSource` instance that still has event handlers attached falls out of use. This example just illustrates how custom event handlers look; it's not a fully engineered solution.)

Example 9-51. Custom `add` and `remove` for sparse events

```
public class ScarceEventSource
{
    // One dictionary shared by all instances of this class,
    // tracking all handlers for all events.
    // Beware of memory leaks - this code is for illustration only.
    private static readonly
        Dictionary<(ScarceEventSource, object), EventHandler> _eventHandlers
        = new();

    // Objects used as keys to identify particular events in the dictionary.
    private static readonly object EventOneId = new();
    private static readonly object EventTwoId = new();

    public event EventHandler EventOne
    {
        add => AddEvent(EventOneId, value);
        remove => RemoveEvent(EventOneId, value);
    }

    public event EventHandler EventTwo
    {
        add => AddEvent(EventTwoId, value);
        remove => RemoveEvent(EventTwoId, value);
    }

    public void RaiseBoth()
    {

```

```

        RaiseEvent(EventOneId, EventArgs.Empty);
        RaiseEvent(EventTwoId, EventArgs.Empty);
    }

    private (ScarceEventSource, object) MakeKey(object eventId) => (this, eventId)

    private void AddEvent(object eventId, EventHandler handler)
    {
        var key = MakeKey(eventId);
        _eventHandlers.TryGetValue(key, out EventHandler? entry);
        entry += handler;
        _eventHandlers[key] = entry;
    }

    private void RemoveEvent(object eventId, EventHandler handler)
    {
        var key = MakeKey(eventId);
        EventHandler? entry = _eventHandlers[key];
        entry -= handler;
        if (entry == null)
        {
            _eventHandlers.Remove(key);
        }
        else
        {
            _eventHandlers[key] = entry;
        }
    }

    private void RaiseEvent(object eventId, EventArgs e)
    {
        var key = MakeKey(eventId);
        if (_eventHandlers.TryGetValue(key, out EventHandler? handler))
        {
            handler(this, e);
        }
    }
}

```

The syntax for custom events is reminiscent of the full property syntax: we add a block after the member declaration that contains the two members, although they are called `add` and `remove` instead of `get` and `set`. (Unlike with properties, you must always supply both methods.) This disables the generation of the field that would normally hold the event,

meaning that the `ScarcEventSource` class has no instance fields at all—instances of this type are as small as it's possible for an object to be.

The price for this small memory footprint is a considerable increase in complexity; I've written about 16 times as many lines of code as I would have needed with compiler-generated events, and we'd need even more to fix the shortcomings described earlier. Moreover, this technique provides an improvement only if the events really are not handled most of the time—if I attached handlers to both events for every instance of this class, the dictionary-based storage would consume more memory than simply having a field for each event in each instance of the class. So you should consider this sort of custom event handling only if you either need nonstandard event-raising behavior or are very sure that you really will be saving memory, and that the savings are worthwhile.

Events and the Garbage Collector

As far as the GC is concerned, delegates are normal objects like any other. If the GC discovers that a delegate instance is reachable, then it will inspect the `Target` property, and whichever object that refers to will also be considered reachable, along with whatever objects that object in turn refers to. Although there is nothing remarkable about this, there are situations in which leaving event handlers attached can cause objects to hang around in memory when you might have expected them to be collected by the GC.

There's nothing intrinsic to delegates and events that makes them unusually likely to defeat the GC. If you do get an event-related memory leak, it will have the same structure as any other .NET memory leak: starting from a root reference, there will be some chain of references that keeps an object reachable even after you've finished using it. Despite this, events often get special blame for memory leaks, and that's because they are often used in ways that can cause problems.

For example, suppose your application maintains some object model representing its state and that your UI code is in a separate layer that makes use of that underlying model, adapting the information it contains for presentation on screen. This sort of layering is usually advisable—it's a bad idea to intermingle code that deals with user interactions and code

that implements the application's logic. But a problem can arise if the underlying model advertises changes in state that the UI needs to reflect. If these changes are advertised through events, your UI code will typically attach handlers to those events.

Now imagine that someone closes one of your application's windows. You would hope that the objects representing that window's UI would all be detected as unreachable the next time the GC runs. The UI framework is likely to have attempted to make that possible. For example, WPF ensures that each instance of its `Window` class is reachable for as long as the corresponding window is open, but once the window has been closed, it stops holding references to the window, to enable all of the UI objects for that window to be collected.

However, if you handle an event from your main application's model with a method in a `Window`-derived class, and if you do not explicitly remove that handler when the window is closed, you will have a problem. As long as your application is still running, something somewhere will presumably be keeping your application's underlying model reachable. This means that the target objects of any delegates held by your application model (e.g., delegates that were added as event handlers) will continue to be reachable, preventing the GC from freeing them. So, if a `Window`-derived object for the now-closed window is still handling events from your application model, that window—and all of the UI elements it contains—will still be reachable and will not be garbage collected.

NOTE

There's a persistent myth that this sort of event-based memory leak has something to do with circular references. In fact, GC copes perfectly well with circular references. It's true that there are often circular references in these scenarios, but they're not the issue. The problem is caused by accidentally keeping objects reachable after you no longer need them. Doing that will cause problems regardless of whether circular references are present.

You can deal with this by ensuring that if your UI layer ever attaches handlers to objects that will stay alive for a long time, you remove those handlers when the relevant UI element is no longer in use. Alternatively, you could use weak references to ensure that if your event source is the only

thing holding a reference to the target, it doesn't keep it alive. WPF can help you with this—it provides a `WeakEventManager` class that allows you to handle an event in such a way that the handling object is able to be garbage collected without needing to unsubscribe from the event. WPF uses this technique itself when databinding the UI to a data source that provides property change notification events.

NOTE

Although event-related leaks often arise in UIs, they can occur anywhere. As long as an event source remains reachable, all of its attached handlers will also remain reachable.

Events Versus Delegates

Some APIs provide notifications through events, while others just use delegates directly. How should you decide which approach to use? In some cases, the decision may be made for you because you want to support some particular idiom. For example, if you want your API to support the asynchronous features in C#, you will need to implement the pattern described in [Chapter 17](#), which uses delegates, but not events, for completion callbacks. Events, on the other hand, provide a clear way to subscribe and unsubscribe, which will make them a better choice in some situations. Convention is another consideration: if you are writing a UI element, events will most likely be appropriate, because that's the predominant idiom.

In cases where constraints or conventions do not provide an answer, you need to think about how the callback will be used. If there will be multiple subscribers for a notification, an event could be the best choice. This is not absolutely necessary, because any delegate is capable of multicast behavior, but by convention, this behavior is usually offered through events. If users of your class will need to remove the handler at some point, events are also likely to be a good choice. That being said, the `IObservable` interface also supports multicast and unsubscription and might be a better choice if you need more advanced functionality. This interface is part of the Reactive Extensions for .NET and is described in [Chapter 11](#).

You would typically pass a delegate as an argument to a method or constructor if it only makes sense to have a single target method. For example, if the delegate type has a non-`void` return value that the API depends on (such as the `bool` returned by the predicate passed to `Array.FindAll`), it makes no sense to have multiple targets or zero targets. An event is the wrong idiom here, because its subscription-oriented model considers it perfectly normal to attach either no handlers or multiple handlers.

Occasionally, scenarios arise in which it might make sense to have either zero handlers or one handler, but never more than one. For example, take WPF's `CollectionView` class, which can sort, group, and filter data from a collection. You configure filtering by providing a `Predicate<object>`. This is not passed as a constructor argument, because filtering is optional, so instead, the class defines a `Filter` property. An event would be inappropriate here, partly because `Predicate<object>` does not fit the usual event delegate pattern, but mainly because the class needs an unambiguous answer of yes or no, so it does not want to support multiple targets. (The fact that all delegate types support multicast means that it's still possible to supply multiple targets, of course. But the decision to use a property rather than an event signals the fact that it's not useful to attempt to provide multiple callbacks here.)

Delegates Versus Interfaces

Back at the start of this chapter, I argued that delegates offer a less cumbersome mechanism for callbacks and notifications than interfaces do. So why do some APIs require callers to implement an interface to enable callbacks? Why do we have `IComparer<T>` and not a delegate? Actually, we have both—there's a delegate type called `Comparison<T>`, which is supported as an alternative by many of the APIs that accept an `IComparer<T>`. Arrays and `List<T>` have overloads of their `Sort` methods that take either.

There are some situations in which the object-oriented approach may be preferable to using delegates. An object that implements `IComparer<T>` could provide properties to adjust the way the comparison works (e.g., the ability to select between various sorting criteria). You may want to

collect and summarize information across multiple callbacks, and although you can do that through captured variables, it may be easier to get the information back out again at the end if it's available through properties of an object.

This is really a decision for whoever is writing the code that is being called back, and not for the developer writing the code that makes the call. Delegates ultimately are more flexible, because they allow the consumer of the API to decide how to structure their code, whereas an interface imposes constraints. However, if an interface happens to align with the abstractions you want, delegates can seem like an irritating extra detail. This is why some APIs present both options, such as the sorting APIs that accept either an `Comparer<T>` or a `Comparison<T>`.

Interfaces might be preferable to delegates if you need to provide multiple related callbacks. The Reactive Extensions for .NET define an abstraction for notifications that includes the ability to know when you've reached the end of a sequence of events or when there has been an error, so in that model, subscribers implement an interface with three methods — `OnNext`, `OnCompleted`, and `OnError`. It makes sense to use an interface, because all three methods are typically required for a complete subscription.

Summary

Delegates are objects that provide a reference to a method, which can be either a static or an instance method. With instance methods, the delegate also holds a reference to the target object, so the code that invokes the delegate does not need to supply a target. Delegates can also refer to multiple methods, although that complicates matters if the delegate's return type is not `void`. While delegate types get special handling from the CLR, they are still just reference types, meaning that a reference to a delegate can be passed as an argument, returned from a method, and stored in a field, variable, or property. A delegate type defines a signature for the target method. This is represented through the type's `Invoke` method, but C# can hide this, offering a syntax in which you can invoke a delegate expression directly without explicitly referring to `Invoke`. You can construct a delegate that refers to any method with a compatible signature.

You can also get C# to do more of the work for you—if you use the lambda syntax to create an anonymous function, C# will supply a suitable declaration for you and can also do work behind the scenes to make variables in the containing method available to the inner one. Delegates are the basis of events, which provide a formalized publish/subscribe model for notifications.

One C# feature that makes particularly extensive use of delegates is LINQ, which is the subject of the next chapter.

- 1 ILDASM ships with Visual Studio. At the time of writing, Microsoft doesn't provide a cross-platform version, but you could use [the open source project ILSpy](#).
- 2 Generic type *definitions* can use the `in` and `out` keywords, but that's different. It indicates when the type *parameter* is contra- or covariant in a generic type. You can't use `in` or `out` when you supply a specific *argument* for a type parameter.
- 3 Alternatively, you may just be one of nature's dynamic language enthusiasts, with an allergy to expressing semantics through static types. If that's the case, C# may not be the language for you.
- 4 `IntPtr` is a value type typically used for opaque handle values, or for working with pointers in low-level high-performance code.
- 5 Unhelpfully, there are two similar terms that mean almost but not quite the same thing. The C# documentation uses *anonymous function* as the general term for either kind of method expression. *Anonymous method* would be a better name for this because these are not all strictly functions—they can have a `void` return—but by the time Microsoft needed a general term for these things, that name was already taken.
- 6 You may be surprised to see `Func<Product, bool>` here and not `Predicate<Product>`. The `Where` method is part of a .NET feature called LINQ that makes extensive use of delegates. To avoid defining huge numbers of new delegate types, LINQ uses `Func` types, and for consistency across the API, it prefers `Func` even when other standard types would fit.